МИНОБРНАУКИ РОССИИ

*Федеральное государственное бюджетное образовательное учреждение высшего образования*
***«МИРЭА – Российский технологический университет»***
**РТУ МИРЭА**

---

Институт радиоэлектроники и информатики
Кафедра геоинформационных систем

**ОТЧЕТ**
**по практической работе № 1**
по дисциплине «МАШИННОЕ ОБУЧЕНИЕ В ОБРАБОТКЕ ПРОСТРАНСТВЕННЫХ ДАННЫХ»

Тема работы: «Построение предсказательной модели на основе парной линейной регрессии»
*(тема работы)*

| | | | |
|---|---|---|---|
| Выполнил: | студент группы РГМО-01-25 | _____ | Ильин К.Л. |
| | *(группа)* | *(подпись)* | *(Фамилия И.О.)* |
| Принял: | Доцент, к.т.н. | _____ | Смольянинова В.А. |
| | *(должность)* | *(подпись)* | *(Фамилия И.О.)* |

Выполнено: «___» _____ 2026 г.

Зачтено: «___» _____ 2026 г.

Москва 2026

# ОГЛАВЛЕНИЕ

**ВВЕДЕНИЕ** ..... 3

**Ход выполнения** ..... 4

**Результаты работы** ..... 9

**Заключение и выводы** ..... 9

**Приложение А** ..... 10

**Приложение Б** ..... 20

# ВВЕДЕНИЕ

**Цель работы**

Изучение процесса построения и работы простейшей модели машинного обучения – предсказательной модели на основе парной линейной регрессии.

**Постановка задачи**

Для исходных данных варианта 1, при помощи метода наименьших квадратов обучить модель парной линейной регрессии двумя способами:

1. путем самостоятельной программной реализации необходимого алгоритма вычислений;
2. путем использования специальных библиотек.

Перед обучением модели проверить данные на наличие выбросов и не учитывать их в процессе обучения.

Оценить качество полученной модели, рассчитав коэффициент детерминации (R2).

Визуализировать анализируемые данные, отметить на них положение выбросов, провести линии регрессии (без учета выбросов), рассчитанные двумя способами.

Сделать предсказание зависимой переменной по заданному значению независимой.

Для выполнения работы использовать средства на основе языка Python.

Подготовить отчет о проделанной работе.

Защитить работу, ответив на вопросы.

## Ход выполнения

### 1–2. Подключение библиотек

Для выполнения работы были импортированы необходимые библиотеки:

```
from sklearn.model_selection import train_test_split
from scipy.optimize import curve_fit
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
import numpy as np
```

### 3. Загрузка данных

Данные персонального варианта были загружены из файла ml1var1.npy с помощью функции np.load(). Матрица содержит пары [x, y] для анализа и последнюю строку [xn, 0], где xn — значение для предсказания.

```
data=np.load("C:/Users/Kirill/Desktop/magsitr/sem2/ML/pr1/ml1var1.npy")
```

### 4. Разделение данных для анализа и предсказания

Исходная матрица разделена на:

- данные для обучения модели;
- значение x = 1.0 для предсказания;

### 5. Обработка выбросов (Z-оценка)

Для обнаружения выбросов использовалась Z-оценка по зависимой переменной y:

$$z(y_i) = \frac{y_i - \bar{y}}{\sigma(y)} \quad \sigma(y) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Точка считалась выбросом, если |z|≥3.

Результаты очистки:

| Показатель | Значение |
|---|---|
| Всего точек | 36 |
| После очистки | 34 |
| Удалено выбросов | 2 |

Выбросы были сохранены отдельно для визуализации.

### 6. Разделение на обучающую и тестовую выборки

Очищенные данные разделены случайным образом с помощью train_test_split:

- Обучающая выборка: 25 точек (~75%)

- Тестовая выборка: 9 точек (~25%)
- random_state = 42 (для воспроизводимости основного эксперимента)

**7–8. Расчёт коэффициентов линейной регрессии**

Модель вида $\hat{y} = kx + b$ обучена двумя способами:

Собственная реализация (формулы МНК):

```
def caluate_k_b(x, y):
    x_mean = np.mean(x)
    y_mean = np.mean(y)
    k = (np.mean(x*y)-x_mean*y_mean)/(np.mean(x**2)-np.mean(x)**2)
    b = np.mean(y) - k*x_mean
    return k, b
```

Библиотечная реализация (scipy.optimize.curve_fit):

```
K, B = caluate_k_b(X_train, Y_train)
curve_K, curve_B = curve_fit(lin_reg, X_train, Y_train)[0]
```

Сравнение результатов:

| Метод | K | B |
|---|---|---|
| Собственная функция | 0.7074293165193083 | 3.74911130319562 |
| curve_fit | 0.7074293166227592 | 3.749111302838584 |
| Разница | $\sim 1.03 \cdot 10^{-10}$ | $\sim 3.57 \cdot 10^{-10}$ |

Результаты совпадают с точностью до машинной погрешности, что подтверждает корректность ручной реализации.

**9. Оценка качества модели ($R^2$)**

Коэффициент детерминации рассчитан на тестовой выборке по формуле:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y})^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

| Метод | $R^2$ |
|---|---|
| Собственная функция | 0.881248 |
| curve_fit | 0.881248 |

Значение $R^2 \approx 0.88$ указывает на высокое качество модели — независимая переменная объясняет ~88% дисперсии зависимой переменной.

**10. Предсказание для заданного x**

Для x = 1.0 выполнено предсказание:

$$\hat{y} = 0.707429 \cdot 1.0 + 3.749111 = 4.456541$$

Оба метода дали идентичный результат (разница $\sim 2.54 \cdot 10^{-10}$).

## 11. Визуализация результатов

Построены графики, отображающие:

- Обучающую выборку (синие точки)
- Выбросы (красные, укрупнённые)
- Тестовую выборку (зелёные, укрупнённые)
- Линию регрессии (собственный расчёт)
- Линию регрессии (curve_fit, пунктир)

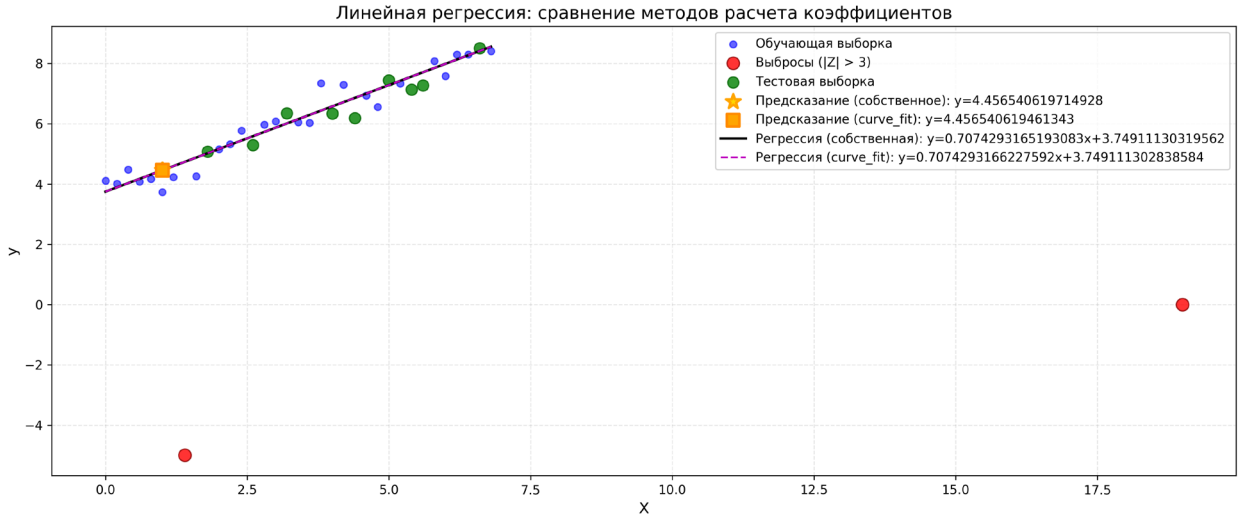


Рисунок 1 – Сравнение методов расчета коэффициентов

Линии регрессии практически сливаются из-за высокой точности совпадения коэффициентов.

## 12. Серия экспериментов с разными random_state

| Эксп. | Seed | K | B | $R^2$ | ŷ(x=1) |
|---|---|---|---|---|---|
| 1 | 42 | 0.707429 | 3.749111 | 0.881248 | 4.456541 |
| 2 | 123 | 0.658284 | 3.877740 | 0.899933 | 4.536024 |
| 3 | 456 | 0.700841 | 3.756525 | 0.955896 | 4.457366 |

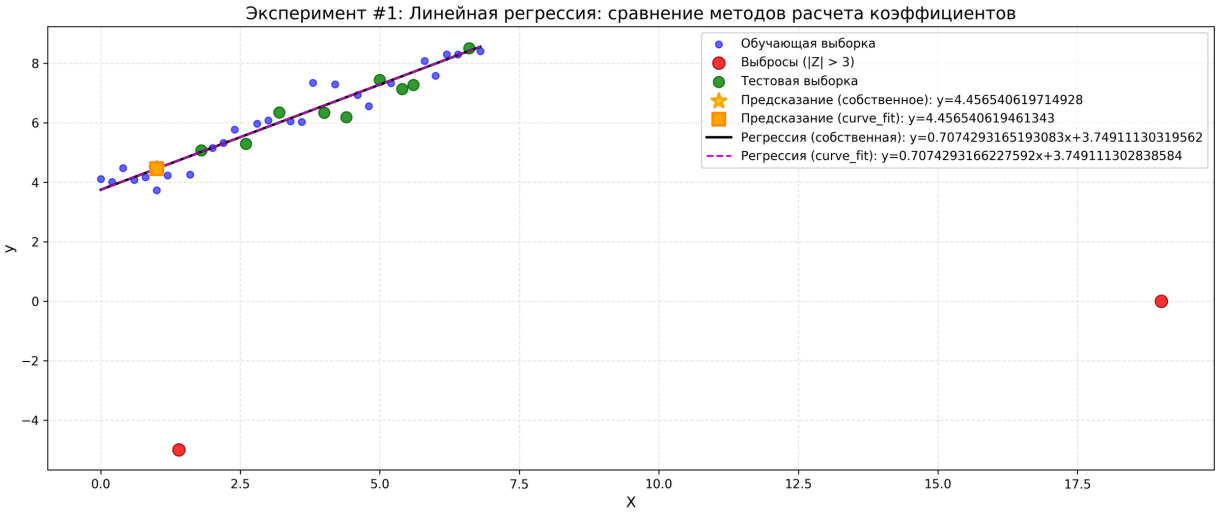


Рисунок 1 – Сравнение методов расчета коэффициентов эксперимент 1

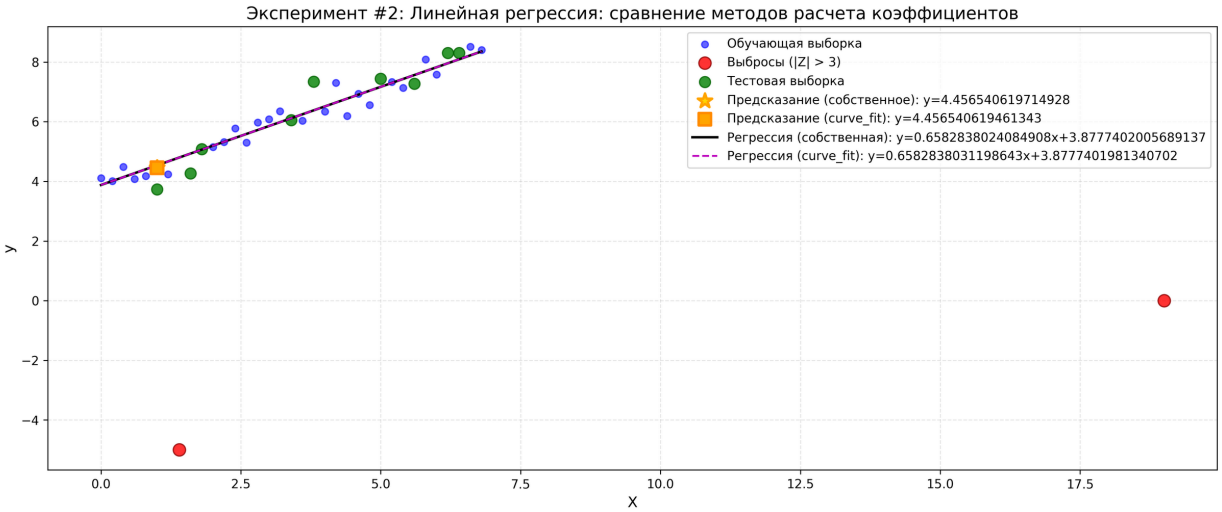


Рисунок 2 – Сравнение методов расчета коэффициентов эксперимент 2

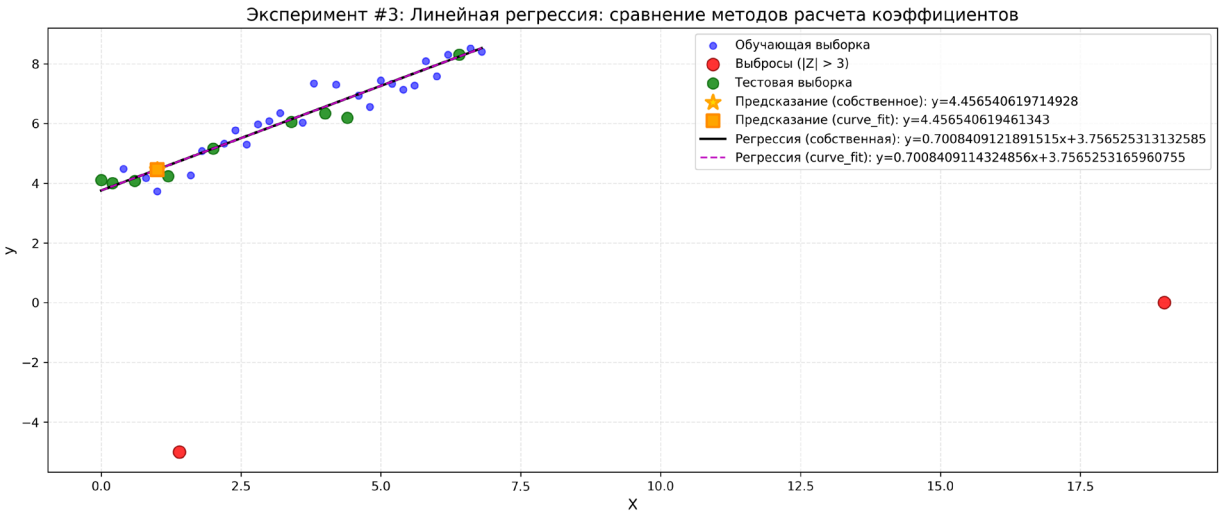


Рисунок 3 – Сравнение методов расчета коэффициентов эксперимент 3

**Объяснение различий:**

1. Случайное разбиение данных формирует разные обучающие/тестовые наборы.
2. МНК чувствителен к составу выборки - разные точки - немного разные коэффициенты.
3. Метрика $R^2$ вычисляется на разных тестовых данных, поэтому также варьируется.
4. При достаточном объёме данных колебания остаются небольшими, что подтверждает устойчивость модели.

## Результаты работы

- Построена предсказательная модель парной линейной регрессии для персонального варианта данных.
- Данные очищены от выбросов методом Z-оценки: удалено 2 аномальных точки из 36.
- Модель обучена двумя способами - ручным и библиотечным. Коэффициенты совпали с точностью до $10^{-10}$.
- Качество модели оценено по $R^2 = 0.881$, что свидетельствует о сильной линейной зависимости.
- Выполнено предсказание: при $x = 1.0$ получено $\hat{y} \approx 4.457$.
- Проведена серия экспериментов, подтвердившая устойчивость модели к вариациям разбиения данных.

## Заключение и выводы

Самостоятельная реализация МНК корректна и даёт результаты, идентичные специализированным библиотекам.

Предварительная очистка от выбросов повышает надёжность модели.

Значение $R^2 > 0.85$ подтверждает пригодность модели для прогнозирования.

Незначительные колебания коэффициентов при разных random_state являются нормальным явлением и подчёркивают важность фиксации случайного генератора для воспроизводимости.

# Приложение А

```
# Построение предсказательной модели
# на основе парной линейной регрессии
from sklearn.model_selection import train_test_split
from scipy.optimize import curve_fit
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
import numpy as np
def calculate_r2_manual(y_true, y_pred):
    """Расчет R2 по формуле: 1 - SS_res / SS_tot"""
    y_mean = np.mean(y_true)
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - y_mean) ** 2)
    if ss_tot == 0:
        return 1.0 if ss_res == 0 else -np.inf
    return 1 - ss_res / ss_tot
def std_deviation(data):
    N = len(data)
    data_mean = np.sum(data)/N
    return np.sqrt(np.sum((data-data_mean)**2)/N)
def Z_estimation(data):
    """Расчет Z-оценок для всех столбцов массива"""
    z_scores = np.zeros_like(data, dtype=float)
    for col in range(data.shape[1]):
        column_data = data[:, col]
        col_mean = np.sum(column_data) / len(column_data)
        std = std_deviation(column_data)
        if std == 0:
            z_scores[:, col] = 0
        else:
```

```
            z_scores[:, col] = (column_data - col_mean) / std
        return z_scores
    def caluate_k_b(x, y):
        x_mean = np.mean(x)
        y_mean = np.mean(y)
        k = (np.mean(x*y)-x_mean*y_mean)/(np.mean(x**2)-np.mean(x)**2)
        b = np.mean(y) - k*x_mean
        return k, b
    def lin_reg(x, k, b):
        return k*x + b
    def predict_y(x_value, k, b):
        """Пункт 10: Предсказание значения y для заданного x"""
        return k * x_value + b
    def plot_regression(X_train, Y_train, X_test, Y_test, X_outliers, Y_out-
liers,K, B, curve_K, curve_B, X, Y, x_for_prediction, y_pred_custom ,
y_pred_curvefit , exp_num=None):
        """
        Пункт 11: Визуализация данных и линий регрессии
        Параметры:
        X_train, Y_train - обучающая выборка
        X_test, Y_test - тестовая выборка
        X_outliers, Y_outliers - выбросы
        K, B - коэффициенты собственной функции
        curve_K, curve_B - коэффициенты curve_fit
        X, Y - все очищенные данные (для масштаба осей)
        exp_num - номер эксперимента (для имени файла)
        """
        # Основной график
        plt.figure(figsize=(14, 6))
        # 1. Обучающая выборка - синие точки
        plt.scatter(X_train, Y_train, c='blue', s=30, alpha=0.6,
```

```
                    label='Обучающая выборка', zorder=2)
        # 2. Выбросы - красные, укрупненные
        if len(X_outliers) > 0:
            plt.scatter(X_outliers, Y_outliers, c='red', s=100, alpha=0.8,
            label='Выбросы (|Z| > 3)', edgecolors='darkred', zorder=3)

        # 3. Тестовая выборка - зеленые, укрупненные
        plt.scatter(X_test, Y_test, c='green', s=80, alpha=0.8,
        label='Тестовая выборка', edgecolors='darkgreen', zorder=4)
        # 4. Предсказанные значения
        # Предсказание собственной функцией - желтая звезда
        plt.scatter(x_for_prediction, y_pred_custom, c='gold', s=150,
marker='*',
        label=f'Предсказание (собственное): y={y_pred_custom}',
        edgecolors='orange', linewidths=2, zorder=9)
        # Предсказание curve_fit - оранжевый квадрат
        plt.scatter(x_for_prediction, y_pred_curvefit, c='orange', s=100,
marker='s', label=f'Предсказание (curve_fit): y={y_pred_curvefit}',
      edgecolors='darkorange', linewidths=2, zorder=10)
        # 5. Линия регрессии (собственная) - черная
        x_line = np.linspace(X.min(), X.max(), 200)
        y_line_custom = lin_reg(x_line, K, B)
        plt.plot(x_line, y_line_custom, 'k-', linewidth=2, label=f'Регрессия
(собственная): y={K}x+{B}', zorder=5)
        # 6. Линия регрессии (curve_fit) - фиолетовая, пунктир
        y_line_curvefit = lin_reg(x_line, curve_K, curve_B)
        plt.plot(x_line, y_line_curvefit, 'm--', linewidth=1.5, la-
bel=f'Регрессия (curve_fit): y={curve_K}x+{curve_B}', zorder=6)
```

```
        plt.xlabel('X', fontsize=12)
        plt.ylabel('y', fontsize=12)
        title = 'Линейная регрессия: сравнение методов расчета коэффициен-
тов'
        if exp_num is not None:
            title = f'Эксперимент #{exp_num}: {title}'
        plt.title(title, fontsize=14)
        plt.legend(fontsize=10, loc='best')
        plt.grid(True, alpha=0.3, linestyle='--')
        plt.tight_layout()
        # Сохранение файла
        if exp_num is not None:
            filename = f'regression_plot_exp{exp_num}.png'
        else:
            filename = 'regression_plot.png'
        plt.savefig(filename, dpi=300, bbox_inches='tight')
        print(f" График сохранен как '{filename}'")
        plt.show()
        # Приближенный просмотр для различения линий
        plt.figure(figsize=(10, 6))
        plt.scatter(X_train, Y_train, c='blue', s=30, alpha=0.5, la-
bel='Train', zorder=2)
        if len(X_outliers) > 0:
            plt.scatter(X_outliers, Y_outliers, c='red', s=100, alpha=0.7,
label='Outliers', zorder=3)
        plt.scatter(X_test, Y_test, c='green', s=80, alpha=0.7, la-
bel='Test', zorder=4)
        plt.plot(x_line, y_line_custom, 'k-', linewidth=3, label='Custom',
zorder=5)
        plt.plot(x_line, y_line_curvefit, 'm--', linewidth=1.5, la-
bel='curve_fit', zorder=6)
```

```
        # Зум на область с данными
        plt.xlim(np.percentile(X, 25), np.percentile(X, 75))
        plt.ylim(np.percentile(Y, 25), np.percentile(Y, 75))
        plt.xlabel('X (приближено)')
        plt.ylabel('y')
        plt.title('Приближенный просмотр: различия линий регрессии')
        plt.legend(fontsize=9)
        plt.grid(True, alpha=0.4, linestyle='--')
        plt.tight_layout()
        if exp_num is not None:
            zoom_filename = f'regression_zoomed_exp{exp_num}.png'
        else:
            zoom_filename = 'regression_zoomed.png'
        plt.savefig(zoom_filename, dpi=300, bbox_inches='tight')
        print(f" Приближенный график сохранен как '{zoom_filename}'")
        plt.show()
    #=========================================================================
======
    # ОСНОВНАЯ ПРОГРАММА
    #=========================================================================
======
    # Загрузка данных
    data = np.load("C:/Users/Kirill/Desktop/magsitr/sem
2/ML/pr1/ml1var1.npy")
    # Удаление выбросов
    z_scores = Z_estimation(data)
    mask = np.all(np.abs(z_scores) <= 3, axis=1)
    data_clean = data[mask]
    # Сохраняем выбросы для визуализации (Пункт 11)
    X_outliers = data[~mask, 0]
    Y_outliers = data[~mask, 1]
```

```
print(f"Всего точек: {len(data)}")
print(f"Осталось после очистки: {len(data_clean)}")
print(f"Удалено выбросов: {len(data) - len(data_clean)}")
X = data_clean[:, 0]
Y = data_clean[:, 1]
# Разделение на выборки
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,
    test_size=0.25,
    random_state=42
)

print(f"\nОбучающая выборка: {len(X_train)} точек")
print(f"Тестовая выборка: {len(X_test)} точек")

# Расчет коэффициентов
K, B = caluate_k_b(X_train, Y_train)
curve_K, curve_B = curve_fit(lin_reg, X_train, Y_train)[0]

print(f"\n[Собственная функция] K = {K}, B = {B}")
print(f"[curve_fit]           K = {curve_K}, B = {curve_B}")

# Предсказания и R2
Y_pred_custom = K * X_test + B
Y_pred_curvefit = curve_K * X_test + curve_B

r2_custom = calculate_r2_manual(Y_test, Y_pred_custom)
r2_curvefit = calculate_r2_manual(Y_test, Y_pred_curvefit)
```

```
print(f"\nR2 (собственная функция): {r2_custom:.6f}")
print(f"R2 (curve_fit):           {r2_curvefit:.6f}")
print(f"Разница между методами:   {abs(r2_custom - r2_curvefit):.2e}")

# ПУНКТ 10: ПРЕДСКАЗАНИЕ ДЛЯ ЗАДАННОГО X
x_for_prediction = 1

y_pred_custom = predict_y(x_for_prediction, K, B)
y_pred_curvefit = predict_y(x_for_prediction, curve_K, curve_B)

print(f"\n{'='*50}")
print(f"ПУНКТ 10: Предсказание для x = {x_for_prediction}")
print(f"{'='*50}")
print(f"Собственная функция: y = {y_pred_custom:.6f}")
print(f"curve_fit:           y = {y_pred_curvefit:.6f}")
print(f"Разница:             {abs(y_pred_custom -
y_pred_curvefit):.2e}")

# ПУНКТ 11: ВИЗУАЛИЗАЦИЯ (основной запуск)
print(f"\n{'='*50}")
print(f"ПУНКТ 11: Визуализация данных")
print(f"{'='*50}")

plot_regression(X_train, Y_train, X_test, Y_test, X_outliers, Y_outli-
ers,
                K, B, curve_K, curve_B, X, Y,x_for_prediction,
y_pred_custom, y_pred_curvefit, exp_num=None)

print(f"\nРазличия в коэффициентах:")
print(f"ΔK = {abs(K - curve_K):.2e}")
print(f"ΔB = {abs(B - curve_B):.2e}")
```

```
    print(f"Линии почти сливаются, так как оба метода решают одну задачу
МНК")

    # ПУНКТ 12: ЭКСПЕРИМЕНТЫ С РАЗНЫМИ random_state
    print(f"\n{'='*50}")
    print(f"ПУНКТ 12: Эксперименты (3 запуска с разными random_state)")
    print(f"{'='*50}")

    results = []

    for exp_num, seed in enumerate([42, 123, 456], 1):
        print(f"\nЭксперимент #{exp_num} (random_state={seed})")

        # Повторное разделение с новым seed
        X_train_exp, X_test_exp, Y_train_exp, Y_test_exp = train_test_split(
            X, Y,
            test_size=0.25,
            random_state=seed,
            shuffle=True
        )

        # Расчет коэффициентов
        k_exp, b_exp = caluate_k_b(X_train_exp, Y_train_exp)
        curve_params_exp, _ = curve_fit(lin_reg, X_train_exp, Y_train_exp)
        curve_k_exp, curve_b_exp = curve_params_exp

        # Предсказание и R2
        Y_pred_exp = k_exp * X_test_exp + b_exp
        r2_exp = calculate_r2_manual(Y_test_exp, Y_pred_exp)
        y_pred_x = predict_y(x_for_prediction, k_exp, b_exp)
```

```
y_pred_custom = predict_y(x_for_prediction, K, B)
y_pred_curvefit = predict_y(x_for_prediction, curve_K, curve_B)

# Сохранение результатов
results.append({
    'experiment': exp_num,
    'random_state': seed,
    'k': k_exp,
    'b': b_exp,
    'curve_k': curve_k_exp,
    'curve_b': curve_b_exp,
    'r2': r2_exp,
    'y_pred': y_pred_x,
    'train_size': len(X_train_exp),
    'test_size': len(X_test_exp),
    'x_for_prediction':x_for_prediction,
    'y_pred_custom':y_pred_custom,
    'y_pred_curvefit':y_pred_curvefit,
    'difference':abs(y_pred_custom - y_pred_curvefit)
})



print(f"   K = {k_exp:.6f}")
print(f"   B = {b_exp:.6f}")
print(f"   R2 = {r2_exp:.6f}")
print(f"   Предсказание y({x_for_prediction}) = {y_pred_x:.6f}")

# Отрисовка графика для каждого эксперимента
```

```
        plot_regression(X_train_exp, Y_train_exp, X_test_exp, Y_test_exp,
                        X_outliers, Y_outliers, k_exp, b_exp,
                        curve_k_exp, curve_b_exp, X, Y,
                        x_for_prediction, y_pred_custom,
y_pred_curvefit,  exp_num=exp_num)


    # Сводная таблица
    print(f"\n{'='*50}")
    print(f"СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ")
    print(f"{'='*50}")
    print(f"{'Эксп.':<6} {'Seed':<6} {'K':<12} {'B':<12} {'R2':<10}
{'y_pred':<12}")
    print("-" * 60)
    for r in results:
        print(f"{r['experiment']:<6} {r['random_state']:<6} {r['k']:<12.6f}
{r['b']:<12.6f} {r['r2']:<10.6f} {r['y_pred']:<12.6f}")


    # Объяснение различий
    print(f"\n{'='*50}")
    print(f"ОБЪЯСНЕНИЕ РАЗЛИЧИЙ МЕЖДУ ЗАПУСКАМИ")
    print(f"{'='*50}")
    print("""
    Почему при разных запусках получаются разные результаты?


    1. Случайное разбиение данных:
       train_test_split с разным random_state формирует разные наборы
       точек в обучающей и тестовой выборках.


    2. Чувствительность метода наименьших квадратов:
       Коэффициенты K и B зависят от конкретных точек в обучающей выборке.
       Разные точки -> немного разные коэффициенты.
```

```
3. Вариативность R2:
   Тестовая выборка тоже меняется, поэтому метрика качества
   вычисляется на разных данных.

4. Стабильность модели:
   При достаточном объеме данных различия должны быть небольшими.
   Если различия большие - возможно, данных мало или есть выбросы.

ВЫВОД:
   Для получения устойчивых оценок рекомендуется:
   - Фиксировать random_state для воспроизводимости
   - Использовать кросс-валидацию вместо одного разбиения
   - Усреднять метрики по нескольким запускам
""")

# Сохранение результатов в файл
import json
with open('experiment_results.json', 'w', encoding='utf-8') as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
print(f"\nРезультаты экспериментов сохранены в
'experiment_results.json'")

print(f"\n{'='*50}")
print(f"ВСЕ ПУНКТЫ ЗАДАНИЯ ВЫПОЛНЕНЫ!")
print(f"{'='*50}")
```

# Приложение Б

```
Всего точек: 36
Осталось после очистки: 34
```

```
Удалено выбросов: 2

Обучающая выборка: 25 точек
Тестовая выборка: 9 точек

[Собственная функция] K = 0.7074293165193083, B = 3.74911130319562
[curve_fit]           K = 0.7074293166227592, B = 3.749111302838584

R2 (собственная функция): 0.881248
R2 (curve_fit):           0.881248
Разница между методами:   4.27e-11

==================================================
ПУНКТ 10: Предсказание для x = 1
==================================================
Собственная функция: y = 4.456541
curve_fit:           y = 4.456541
Разница:             2.54e-10

==================================================
ПУНКТ 11: Визуализация данных
==================================================
 График сохранен как 'regression_plot.png'
 Приближенный график сохранен как 'regression_zoomed.png'

Различия в коэффициентах:
ΔK = 1.03e-10
ΔB = 3.57e-10
Линии почти сливаются, так как оба метода решают одну задачу МНК

==================================================
ПУНКТ 12: Эксперименты (3 запуска с разными random_state)
==================================================

Эксперимент #1 (random_state=42)
   K = 0.707429
   B = 3.749111
   R2 = 0.881248
   Предсказание y(1) = 4.456541
 График сохранен как 'regression_plot_exp1.png'
 Приближенный график сохранен как 'regression_zoomed_exp1.png'
```

```
Эксперимент #2 (random_state=123)
   K = 0.658284
   B = 3.877740
   R2 = 0.899933
   Предсказание y(1) = 4.536024
 График сохранен как 'regression_plot_exp2.png'
 Приближенный график сохранен как 'regression_zoomed_exp2.png'

Эксперимент #3 (random_state=456)
   K = 0.700841
   B = 3.756525
   R2 = 0.955896
   Предсказание y(1) = 4.457366
 График сохранен как 'regression_plot_exp3.png'
 Приближенный график сохранен как 'regression_zoomed_exp3.png'

==================================================
СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ
==================================================
Эксп.  Seed   K            B            R2         y_pred
------------------------------------------------------------
1      42     0.707429     3.749111     0.881248   4.456541
2      123    0.658284     3.877740     0.899933   4.536024
3      456    0.700841     3.756525     0.955896   4.457366

==================================================
ОБЪЯСНЕНИЕ РАЗЛИЧИЙ МЕЖДУ ЗАПУСКАМИ
==================================================

Почему при разных запусках получаются разные результаты?

1. Случайное разбиение данных:
   train_test_split с разным random_state формирует разные наборы
   точек в обучающей и тестовой выборках.

2. Чувствительность метода наименьших квадратов:
   Коэффициенты K и B зависят от конкретных точек в обучающей выборке.
   Разные точки -> немного разные коэффициенты.

3. Вариативность R2:
   Тестовая выборка тоже меняется, поэтому метрика качества
   вычисляется на разных данных.
```

```
4. Стабильность модели:
   При достаточном объеме данных различия должны быть небольшими.
   Если различия большие - возможно, данных мало или есть выбросы.

ВЫВОД:
   Для получения устойчивых оценок рекомендуется:
   - Фиксировать random_state для воспроизводимости
   - Использовать кросс-валидацию вместо одного разбиения
   - Усреднять метрики по нескольким запускам


Результаты экспериментов сохранены в 'experiment_results.json'

==================================================
ВСЕ ПУНКТЫ ЗАДАНИЯ ВЫПОЛНЕНЫ!
==================================================
```